

Lecture 27: Diffusion I

Destroying an image on purpose

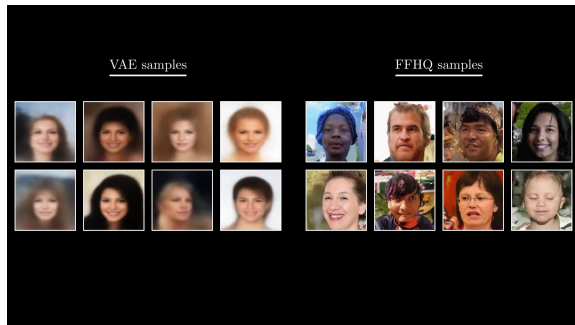
Figures marked as such are stills from Welch Labs / 3Blue1Brown (2025) and Deepia (2025).

Two lectures ago I owed you an explanation

In L23 we built the variational autoencoder, and ended on an honest admission: the samples are **blurry**. The closing slide named something better and did not explain it.

On the right: the same dataset of faces, same resolution. Left half generated by a diffusion model, right half by a VAE.

This chapter is the explanation. By L31 you will know what every text-to-image model on your phone is actually doing.

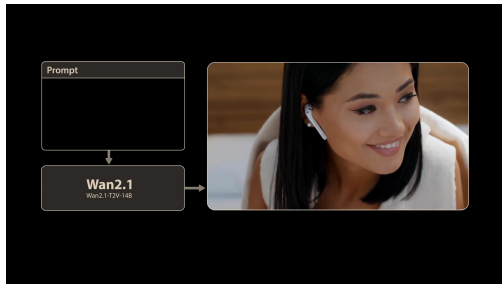


Deepia (2025)

Where does a generated video begin?

A prompt goes into an open-source video model. Out comes this.

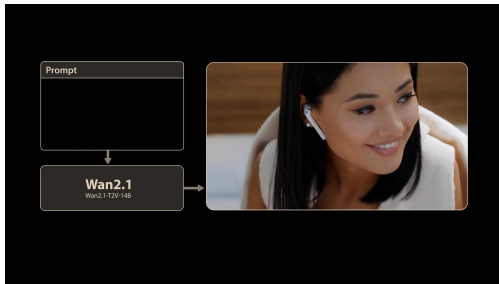
Predict first: what does the model start from?



Welch Labs / 3Blue1Brown (2025)

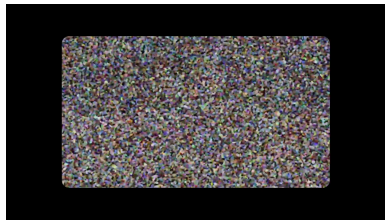
Where does a generated video begin?

A prompt goes into an open-source video model. Out comes this.



Welch Labs / 3Blue1Brown (2025)

Predict first: what does the model start from?



A call to a **random number generator**. Every pixel independent noise. The model is then run about **50 times**, each pass nudging the noise a little closer to a video.

Outline

Generation is sampling

The forward process

Turning around

Generation is sampling

We want to draw from a distribution nobody can write down.

The problem, stated honestly

Call $p(\mathbf{x})$ the distribution of **natural images**: the tiny corner of $\mathbb{R}^{512 \times 512 \times 3}$ where photographs live, and the vast rest is static.

Generating an image = drawing a fresh sample from $p(\mathbf{x})$.

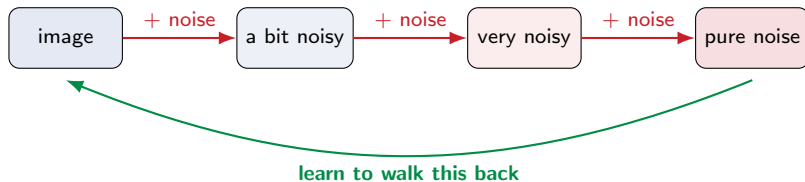
But we have no formula for $p(\mathbf{x})$, and there probably is not one. All we have is a pile of samples: the training set.

So: **sample from a distribution you cannot write down, evaluate, or normalize.** Every generative model is an answer to this one question.

L23's answer: learn a decoder from a simple latent. It worked, but blurred.

The diffusion bet

Instead of modelling $p(\mathbf{x})$ directly, destroy it in a way we control completely.



The forward direction is **ours**: we choose the noise, so we know it exactly. The reverse direction is the only thing we have to learn.

Sohl-Dickstein et al. (2015) had this idea and the name. It sat mostly ignored for five years while everyone worked on GANs.

The forward process

Break it correctly, or the whole scheme fails before training starts.

One step of noising

Write $\mathbf{x}_0$ for a clean image. One step adds Gaussian noise scaled by β :

$$\mathbf{x}_1 = \mathbf{x}_0 + \sqrt{\beta} \varepsilon, \quad \varepsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

Equivalently, and more usefully:

$$q(\mathbf{x}_1 | \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_0, \beta \mathbf{I})$$

"Adding noise to $\mathbf{x}_0$ " and "sampling from a Gaussian centred on $\mathbf{x}_0$ " are the same sentence. The second form is what lets us do algebra later.

Independent Gaussian noise **adds in variance**. Two steps of β is one step of 2β :

$$q(\mathbf{x}_t | \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_0, t\beta \mathbf{I})$$

So we can jump straight to step t without simulating the ones in between.

Does that actually reach pure noise?

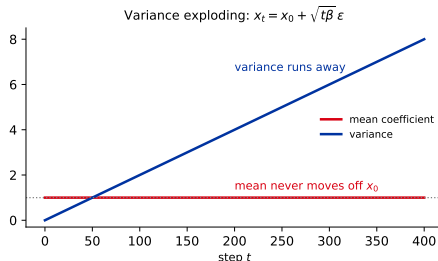
We wanted the forward process to end at $\mathcal{N}(\mathbf{0}, \mathbf{I})$, so that at generation time we can start from a random draw.

Predict first: with $q(\mathbf{x}_t \mid \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_0, t\beta\mathbf{I})$, does it?

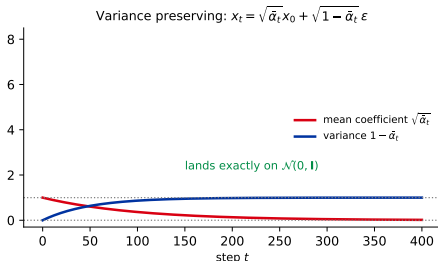
Does that actually reach pure noise?

We wanted the forward process to end at $\mathcal{N}(\mathbf{0}, \mathbf{I})$, so that at generation time we can start from a random draw.

Predict first: with $q(\mathbf{x}_t | \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_0, t\beta\mathbf{I})$, does it?



No. The mean never leaves $\mathbf{x}_0$ and the variance grows without bound. This is called **variance exploding** (VE).



You never arrive at a fixed distribution, so there is nothing to sample from when you want to run the process backwards.

The fix: shrink the mean as you add noise

Scale the previous image down by $\sqrt{1 - \beta}$ before adding the noise:

$$\mathbf{x}_t = \sqrt{1 - \beta} \mathbf{x}_{t-1} + \sqrt{\beta} \varepsilon$$

Name the surviving fraction of each step $\alpha_t := 1 - \beta_t$, and its running product $\bar{\alpha}_t := \prod_{s=1}^t \alpha_s$. Unrolling back to $\mathbf{x}_0$ then gives the formula we will use constantly:

$$q(\mathbf{x}_t | \mathbf{x}_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I}) \quad \text{with} \quad \bar{\alpha}_t = (1 - \beta)^t \text{ for constant } \beta$$

Three symbols, and you need all of them: β_t = noise added at step t ; $\alpha_t = 1 - \beta_t$ = signal kept at step t ; $\bar{\alpha}_t$ = signal kept *in total*. Note $\bar{\alpha}_t = \alpha_t \bar{\alpha}_{t-1}$.

Since $0 < \beta < 1$, we get $\bar{\alpha}_t \rightarrow 0$. So the mean coefficient $\sqrt{\bar{\alpha}_t} \rightarrow 0$ and the variance $1 - \bar{\alpha}_t \rightarrow 1$: the process lands exactly on $\mathcal{N}(\mathbf{0}, \mathbf{I})$. This is **variance preserving (VP)**.

Two things to keep straight:

- ▶ $\bar{\alpha}_t = (1 - \beta)^t$ holds for **constant** β . In a moment β will vary and it becomes a product.
- ▶ "Variance preserved at 1" assumes the data was scaled to $[-1, 1]$ first. DDPM does exactly that.

Before we move on: variance exploding is not a mistake

We discarded VE because it did not suit us, not because it is wrong.

VE is a real, working family. Score matching (Song & Ermon, 2019) is built on it, and Song et al. (2021) showed VE and VP are two members of one continuous family.

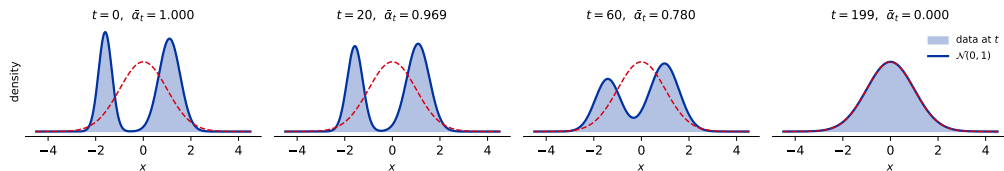
Why DDPM picked VP: it converges to a *fixed* $\mathcal{N}(\mathbf{0}, \mathbf{I})$ that you can draw from directly. With VE you must track how wide the noise has become.

DDPM is Denoising Diffusion Probabilistic Models (Ho, Jain & Abbeel, 2020): the paper that took diffusion from a curiosity to the state of the art, and the spine of the next two lectures.

Watch it happen

A deliberately awkward data distribution, pushed through the VP forward process:

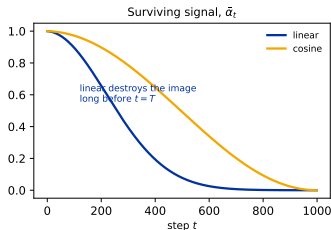
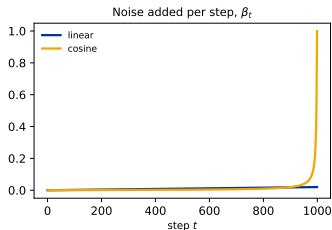
The forward process erases structure: any $p(x)$ becomes $\mathcal{N}(0, 1)$



Nothing here is learned. Two bumps of unequal mass and width are dragged toward the origin and flattened, and by $t = 199$ the red dashed $\mathcal{N}(0, 1)$ sits exactly on top. **Whatever your data was, the forward process forgets it.**

Noise schedules: β does not have to be constant

In practice β varies with t , and then $\bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s)$.



By hand: constant $\beta = 0.02$

t	$\bar{\alpha}_t = 0.98^t$	$\sqrt{\bar{\alpha}_t}$
10	8.17e-01	0.9039
100	1.33e-01	0.3642
500	4.10e-05	0.0064
1000	1.68e-09	0.0000

Linear (Ho et al., 2020) destroys the image well before $t = T$, so the late steps teach the model almost nothing.

Cosine (Nichol & Dhariwal, 2021) holds signal longer and spends the schedule where there is still something to learn.

By hand: how fast does the image actually die?

Take a constant $\beta = 0.02$, so $\bar{\alpha}_t = 0.98^t$, and the surviving fraction of the original image is $\sqrt{\bar{\alpha}_t}$.

t	$\bar{\alpha}_t = 0.98^t$	$\sqrt{\bar{\alpha}_t}$	what is left
10	$0.98^{10} = 0.817$	0.904	barely touched
100	$0.98^{100} = 0.133$	0.364	recognizable, very grainy
500	$0.98^{500} = 4.1 \times 10^{-5}$	0.0064	essentially gone
1000	$0.98^{1000} = 1.7 \times 10^{-9}$	0.00004	noise

The decay is **geometric**, so "how many steps" is much less interesting than "what is $\bar{\alpha}_t$ ". Papers quote $\bar{\alpha}$, not t , for this reason.

Note $\sqrt{\bar{\alpha}_{1000}} \approx 4 \times 10^{-5}$: at that point $\mathbf{x}_T$ really is a draw from $\mathcal{N}(\mathbf{0}, \mathbf{I})$, to any precision that matters.

Why the closed form matters so much

To train on step t , the naive route is to simulate:

```
x = x0
```

```
for s in range(t): x = step(x)
```

That is t operations per training example, with t up to 1000.

The closed form does it in **one**:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon$$

This is the frame to remember. Pick a random t , draw one ε , get $\mathbf{x}_t$ immediately. It is what makes diffusion training cheap enough to be practical, and you will see it again in every equation for the next two lectures.

Turning around

Destroying was free. Now the part that needs a network.

The reverse step is where the parameters live

The forward process q has **no learned parameters at all** - just β and a random draw. Everything we do not know sits in the reverse direction:

$$p_{\theta}(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$$

θ are the weights of a neural network. If we can learn one reverse step, we can chain it: start at $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and run down to $\mathbf{x}_0$.

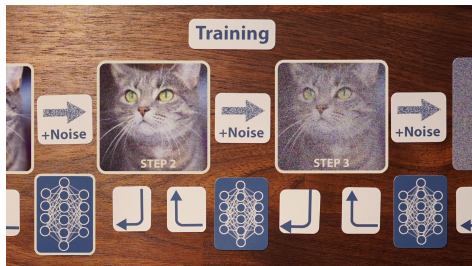
Note the asymmetry: q needs $\mathbf{x}_0$ to be defined, p_{θ} must not - at generation time $\mathbf{x}_0$ is exactly the thing we are trying to produce. This tension drives the whole of next lecture.

The obvious plan, and why it is not the one used

Predict first: train the network to undo one step - given $\mathbf{x}_t$, predict $\mathbf{x}_{t-1}$. Then chain it T times. What goes wrong?

The obvious plan, and why it is not the one used

Predict first: train the network to undo one step - given $\mathbf{x}_t$, predict $\mathbf{x}_{t-1}$. Then chain it T times. What goes wrong?



Welch Labs / 3Blue1Brown (2025)

It **does not work well**, and essentially no modern model is built this way.

The single step from $\mathbf{x}_t$ to $\mathbf{x}_{t-1}$ is almost entirely the random draw we just made. Asking a network to predict it is asking it to predict noise from noise: the target is nearly pure variance, and the learnable part is a thin signal underneath.

There is a target with the *same* information and far less variance. Finding it is what the next lecture derives.

Recap

1. Generating = sampling from a $p(\mathbf{x})$ we cannot write down.
2. Diffusion destroys the data on purpose, then learns the way back.
3. Adding noise naively makes the variance explode and never reaches a fixed distribution.
4. Shrinking the mean by $\sqrt{1-\beta}$ fixes it: the variance-preserving process lands on $\mathcal{N}(\mathbf{0}, \mathbf{I})$.
5. $\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\varepsilon$ gives any step in one shot.

Next: we know how to break an image, and we know that "undo one step" is the wrong training target.

L28 starts from maximum likelihood, follows the algebra where it goes, and lands on a loss function that fits on one line - and is just a mean squared error.

One prompt to hold onto: *"a lone tree standing in the middle of a desert, casting a long shadow on the sand."* We come back to it in every remaining lecture - it goes blurry in L29, loses its tree in L30, and gets cheap in L31.